

Zadanie 6 - panel logowania Helpdesk z wykorzystaniem AI

Dokument opisuje proces użycia AI jako junior developera do wygenerowania panelu logowania, code review oraz poprawioną wersję produkcyjną. System jest zgodny z wcześniejszym Helpdeskiem sklepu internetowego: są role użytkowników, backend odpowiada za kontrolę dostępu, hasła są chronione, a aplikacja ma ograniczać brute force i przeciążenie.

1. Pierwotna wersja od AI

Prompt wpisany do AI:

```
Napisz prosty panel logowania do systemu Helpdesk w Python Flask.  
Ma być formularz logowania, baza SQLite z użytkownikami i po zalogowaniu panel użytkownika.  
Kod ma być prosty, żeby dało się go szybko uruchomić lokalnie.
```

Rezultat: AI wygenerowało prosty panel logowania we Flasku z formularzem, bazą SQLite i dwoma użytkownikami testowymi. Kod działał jako demonstracja, ale był typowo developerski: bez hashy haseł, bez bezpiecznej sesji, bez rate limitingu i z zapytaniem SQL składanym przez konkatenację tekstu.

2. Security code review pierwotnego kodu

Luka	STRIDE	CIA	Konsekwencja	Fragment kodu
Hasła zapisane jawnym tekstem	Information Disclosure	Poufność	Po wycieku bazy atakujący od razu zna hasła użytkowników.	INSERT ... 'admin123' / 'klient123'
SQL Injection w logowaniu	Tampering / Elevation of Privilege	Poufność, integralność	Atakujący może ominąć logowanie albo odczytać dane, jeśli poda spreparowany email lub hasło.	query = "SELECT ... "" + email + "" ..."
Brak rate limitingu	Denial of Service / Spoofing	Dostępność, poufność	Można automatycznie zgadywać hasła albo przeciążyć endpoint /login.	brak limitu w route /login
Brak prawdziwej sesji	Spoofing	Poufność	Aplikacja nie tworzy bezpiecznego tokenu ani HttpOnly cookie, więc nie ma kontroli sesji.	return render_template_string(PANEL_HTML...)
Brak walidacji inputu	Injection / Tampering	Integralność	Dane z formularza trafiają bez sprawdzenia do zapytania SQL i logiki aplikacji.	email = request.form.get("email")
Debug mode w aplikacji	Information Disclosure	Poufność	W razie błędu debug może ujawnić szczegóły aplikacji i środowiska.	app.run(debug=True)

3. Poprawiona wersja produkcyjna

Problem	Poprawka	Dlaczego tak
Hasła jawne	bcrypt.hashpw() przy tworzeniu użytkownika i bcrypt.checkpw() przy logowaniu.	Nawet po wycieku bazy hasła nie są zapisane wprost.
Brak sesji	JWT z datą wygaśnięcia zapisany w HttpOnly cookie.	Token nie jest trzymany w localStorage, więc JavaScript nie ma do niego bezpośredniego dostępu.
Brute force	Prosty rate limiting: maksymalnie 5 prób logowania z IP na minutę.	Adresuje wcześniejsze wymaganie ograniczania prób logowania i ryzyko brute force.
SQL Injection	Zapytania SQLite z placeholderami, np. WHERE email = ?.	Parametryzacja oddziela dane użytkownika od treści zapytania SQL.
Brak walidacji	Sprawdzenie formatu e-maila, długości hasła i pustych pól.	Niepoprawne dane są odrzucane przed logiką logowania.
HTTPS	Komentarz i flaga FORCE_HTTPS ustawiająca Secure cookie w produkcji.	Lokalnie można testować po HTTP, a w produkcji cookie powinno iść tylko po HTTPS.
Brak audytu	Tabela security_log zapisuje logowania udane, błędne i rate limit.	Pomaga później sprawdzić, co działo się w systemie.

4. Co LLM zrobił źle

Największy błąd LLM polegał na tym, że wygenerował kod działający, ale niebezpieczny. Skupił się na tym, żeby formularz przyjmował login i hasło, a nie na tym, jak te dane chronić. To jest dokładnie ryzyko z wcześniejszej analizy: backend musi wymuszać bezpieczeństwo, bo same elementy interfejsu nie wystarczają.

LLM użył też najprostszych rozwiązań: jawnych haseł, konkatencji SQL i braku sesji. W poprawionej wersji te miejsca zostały zastąpione mechanizmami, które wcześniej pojawiły się w wymaganiach bezpieczeństwa: bcrypt, kontrola sesji, rate limiting, walidacja i parametryzowane zapytania.

5. Jak uruchomić kod

```
cd secure_app
python -m venv .venv
# Windows:
.venv\Scripts\activate
# Linux/macOS:
source .venv/bin/activate

pip install -r requirements.txt
python app.py

Adres: http://127.0.0.1:5000
Konta:
admin@sklep.pl / Admin123!
klient@sklep.pl / Klient123!
```

6. Fragmenty poprawionego kodu

Hashowanie hasła bcrypt:

```
password_hash = bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt())
bcrypt.checkpw(password.encode("utf-8"), user["password_hash"])
```

JWT w HttpOnly cookie:

```
response.set_cookie(
```

```
    "session_token",  
    token,  
    httponly=True,  
    secure=FORCE_HTTPS,  
    samesite="Strict",  
    max_age=JWT_TTL_MINUTES * 60,  
)
```

Parametryzowane zapytanie:

```
user = db.execute(  
    "SELECT id, email, password_hash, role FROM users WHERE email = ?",  
    (email,),  
).fetchone()
```